Documentación de las Validaciones de PDF

Investigacion de la implementacion utilizando `PyMuPDF` . Libreria recomendada para nuestro caso.

PyMuPDF (fitz)

Cubre nuestras 7 validaciones con una sola dependencia. Muy rapida (ya que utiliza C), permite detectar cifrado, leer metadatos, extraer texto por pagina y verificar el header binario del archivo.

Agregar al proyecto con:
`uv add pymupdf`

VALIDACIONES:

1. **¿Existe el archivo?**

   FastAPI provee el objeto `UploadFile`. Si el campo es requerido y no viene nada, FastAPI ya lanza un 422. Pero si viene vacío o con filename null, hay que chequearlo explícitamente.

   ```
   from fastapi import UploadFile, HTTPException

   async def validate_exists(file: UploadFile):
       if not file or not file.filename:
           raise HTTPException(
               status_code=400,
               detail="No se recibió ningún archivo."
           )
   ```

Excepción a lanzar: `HTTP 400`

2. **¿Pesa más de 0 KB?**

Se lee el contenido en memoria con `await file.read()` y se verifica la longitud. Este paso también es necesario para las validaciones posteriores de header y fitz.

```python
async def validate_size(content: bytes):
    if len(content) == 0:
        raise HTTPException(
            status_code=400,
            detail="El archivo está vacío (0 KB)."
        )
    # Opcional: límite máximo (ej. 10MB)
    MAX_SIZE = 10 * 1024 * 1024
    if len(content) > MAX_SIZE:
        raise HTTPException(status_code=413, detail="Archivo
demasiado grande.")
```

3. **¿Extensión .pdf?**

Primera línea de defensa. Rápida y simple. No es suficiente sola (se puede renombrar cualquier archivo), pero filtra errores comunes del usuario.

```python
import os

def validate_extension(filename: str):
    ext = os.path.splitext(filename)[1].lower()
    if ext != ".pdf":
        raise HTTPException(
            status_code=400,
            detail=f"Extensión inválida '{ext}'. Se requiere
.pdf"
        )
```

4. **¿Header binario %PDF-?**

Un archivo renombrado como .pdf pero que es en realidad una imagen o un ZIP no va a tener este magic number. Esta validación detecta archivos falsos antes de intentar parsearlos.

```python
def validate_header(content: bytes):
    # Los primeros 5 bytes deben ser %PDF-
    if not content.startswith(b"%PDF-"):
        raise HTTPException(
            status_code=400,
            detail="El archivo no es un PDF válido (magic num
ber inválido)."
        )
```

Nota: El estándar PDF especifica que el header `%PDF-` debe aparecer en los primeros 1024 bytes, aunque casi siempre está en el byte 0.
Chequear `content[:1024]` es más tolerante.

5. **¿Está cifrado?**

PDFs cifrados con contraseña no pueden ser leídos sin ella. PyMuPDF lanza una excepción al intentar abrir uno, o podés consultarlo directamente con `doc.needs_pass`.

```python
import pymupdf

def validate_not_encrypted(content: bytes) -> pymupdf.Documen
t:
    doc = pymupdf.open(stream=content, filetype="pdf")
    if doc.needs_pass:
        doc.close()
        raise HTTPException(
            status_code=422,
            detail="El PDF está cifrado con contraseña."
        )
    return doc  # retornamos el doc abierto para reusar
```

Abrí el documento una sola vez y pasá el objeto `doc` a las validaciones siguientes. Abrir y cerrar fitz por cada check es innecesariamente costoso.

6. **¿Tiene páginas?**

Un PDF puede ser técnicamente válido pero tener 0 páginas (PDFs corruptos o mal generados). `doc.page_count` lo revela inmediatamente.

```
def validate_has_pages(doc: fitz.Document):
    if doc.page_count == 0:
        doc.close()
        raise HTTPException(
            status_code=422,
            detail="El PDF no contiene páginas."
        )
```

7. **¿Tiene texto extraíble?**

PDFs escaneados son imágenes dentro de un PDF — no tienen texto digital. Como tu app extrae texto, esta es la validación más importante para la lógica de negocio. Se itera sobre las páginas buscando al menos algo de texto.

```
def validate_has_text(doc: fitz.Document):
    has_text = False
    for page in doc:
        text = page.get_text().strip()
        if len(text) > 10:  # umbral mínimo
            has_text = True
            break
    if not has_text:
        doc.close()
        raise HTTPException(
            status_code=422,
            detail="El PDF no contiene texto extraíble. ¿Es u
n PDF escaneado?"
        )
```

El umbral de `> 10` caracteres evita falsos negativos por PDFs con solo whitespace. Podés ajustarlo según tu criterio.